RAS 585: Machine Learning and Artificial Intelligence
Assignment 2 Report

Hunter Flechsig

Fall 2026

1 Overview

This assignment explores machine learning methods for image compression, density estimation, classification, and object recognition. First, agglomerative hierarchical clustering and k-means are used for image compression and compared using reconstruction error and compression rate.

Next, the Iris dataset is analyzed with a four-dimensional Gaussian kernel density estimate and three classifiers: k-nearest neighbors (KNN), a classification tree, and Linear Discriminant Analysis (LDA). Their prediction accuracy, confusion matrices, and classification mechanisms are compared.

Finally, a pretrained YOLO model is applied to an image for object recognition and localization, followed by a comparison of convolutional neural networks (CNNs) and multilayer perceptrons (MLPs).

Note: Full code, intermediate calculations, plots, and program outputs are provided in the submitted Jupyter notebook and notebook PDF. This report presents the main results and findings.

2 Part 1: Image Compression

In this section, two different clustering methods are applied to compress the coffee image shown in Figure 1. For each method, the image is divided into non-overlapping $c \times c$ pixel windows using window sizes of 20, 50, and 80. Since these window sizes do not always divide the image dimensions evenly, reflection padding is added to ensure that the entire padded image can be divided into equal-sized windows. Each RGB window is then flattened into a single vector and treated as one sample for clustering. The windows are clustered using k values of 6, 10, and 14, resulting in nine combinations of k and c for each clustering method.



Figure 1: Original coffee image used for the image compression experiments.

1. Image Compression with Hierarchical Clustering

The first compression technique uses agglomerative hierarchical clustering with single linkage. Agglomerative clustering begins with each image window as its own cluster. At each step, the two clusters with the smallest distance between any pair of their samples are merged. This process continues until the desired number of k clusters remains. Each image window is then represented by the reference vector associated with its assigned cluster. The compressed image is reconstructed by replacing each window with its corresponding reference vector.

Figure 2 shows the reconstructed images for all nine combinations of k and c.

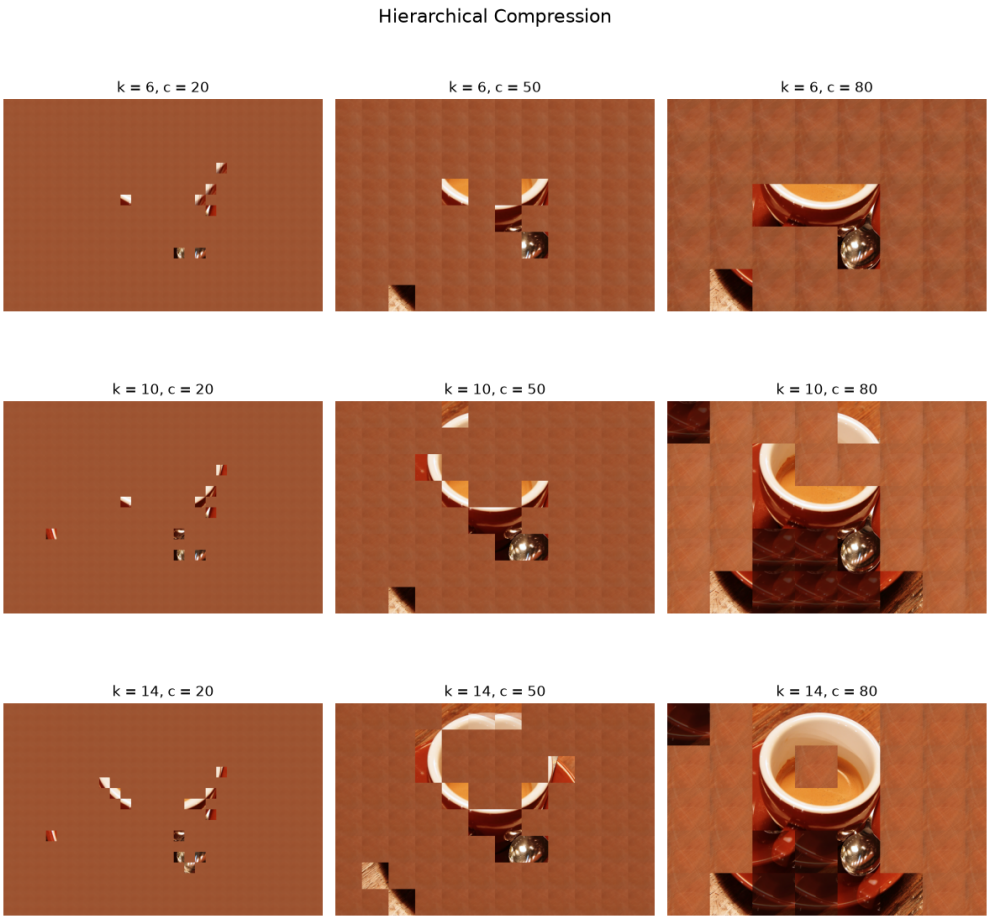


Figure 2: Hierarchical clustering reconstructions for each combination of cluster count k and window size c.

2. Analysis of Hierarchical Clustering

The hierarchical clustering configurations were compared using reconstruction error and compression rate. Lower reconstruction error indicates better image quality, while a lower compression rate indicates greater compression. To compare these competing objectives, MSE and compression rate were each scaled by their maximum observed value and combined into an equal-weight score:

$$\text{score} = \frac{\text{MSE}}{\max(\text{MSE})} + \frac{\text{compression rate}}{\max(\text{compression rate})}.$$

A lower score indicates a better trade-off under this custom metric. Figure 3 shows the two scaled values, where points closer to the bottom-left are preferred. Figure 4 shows that $k = 6$ and $c = 20$ gives the lowest equal-weight score. A different weighting could produce a different preferred configuration.

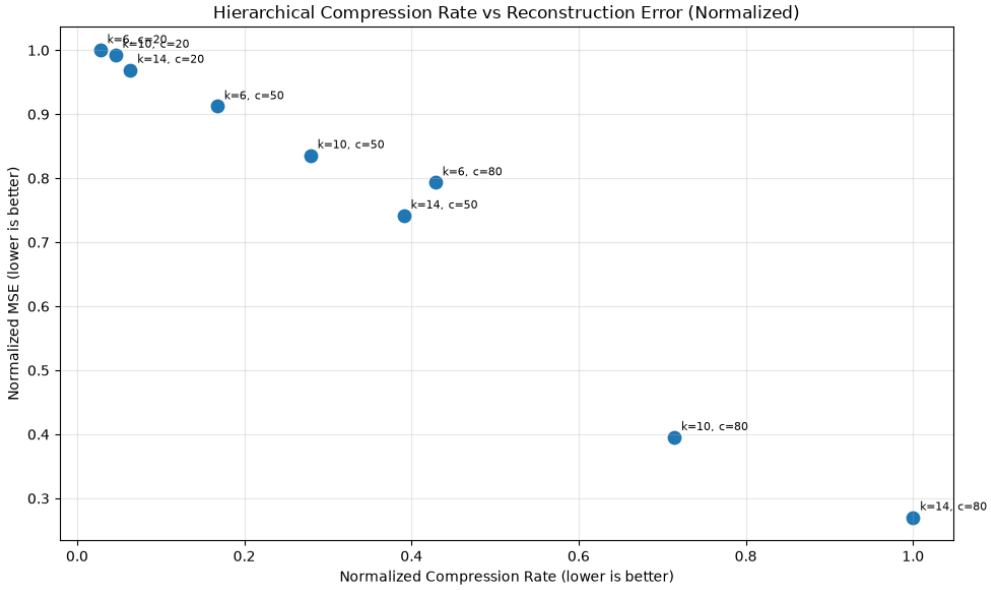


Figure 3: Scaled reconstruction error and compression rate for the hierarchical clustering configurations. Points closer to the bottom-left represent a better trade-off between the two objectives.

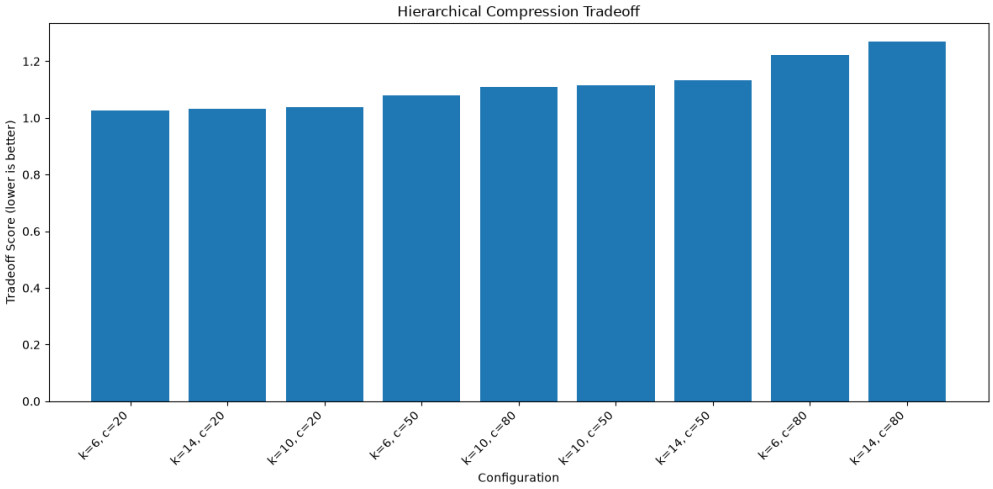


Figure 4: Equal-weight trade-off scores for the hierarchical clustering configurations. Lower scores indicate a better balance between reconstruction error and compression rate.

3. Image Compression with K-means Clustering

The second compression technique uses k-means clustering. K-means initializes k cluster centers, assigns each image window to its nearest center, and updates each center using the mean of its assigned samples. This repeats until the assignments stabilize. The image is then reconstructed using the reference vector for each window's assigned cluster.

The same values of $k \in \{6, 10, 14\}$ and $c \in \{20, 50, 80\}$ were tested. Figure 5 shows the reconstructed images for all nine combinations.

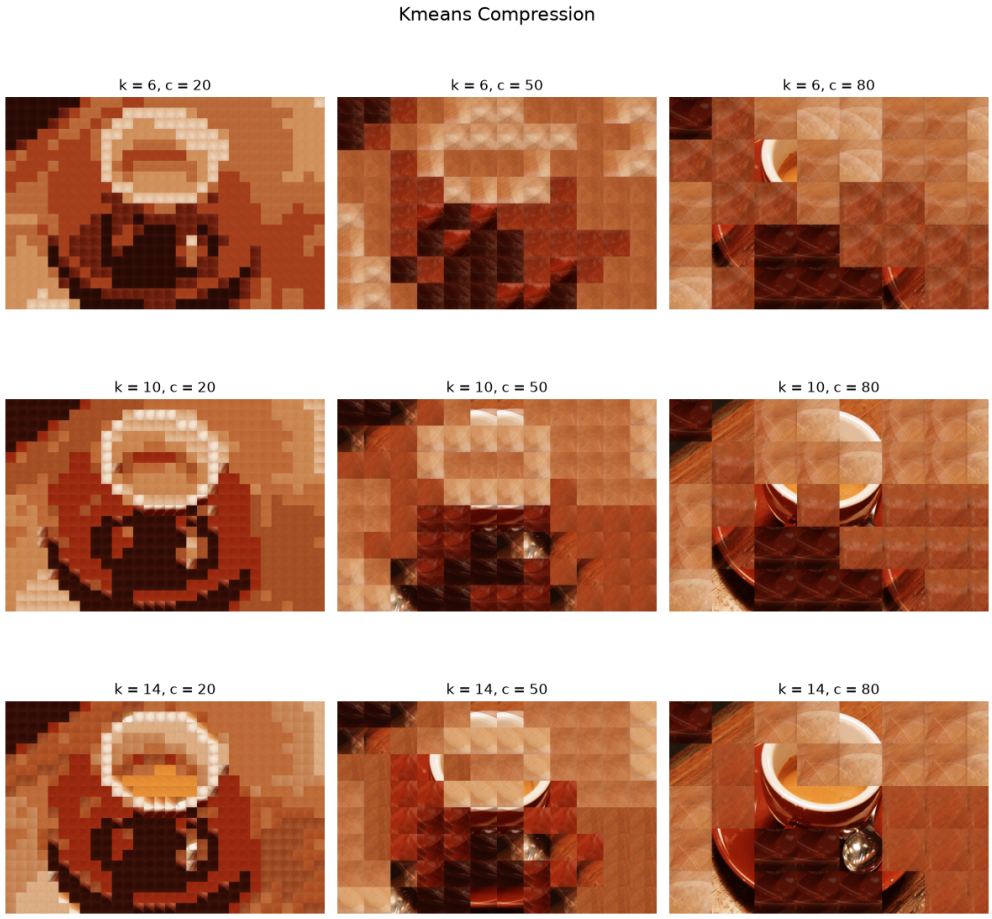


Figure 5: K-means reconstructions for each combination of cluster count k and window size c.

4. Analysis and Comparison

The same scoring method used for hierarchical clustering was applied to the k-means results. Based on this score, $k = 14$ and $c = 20$ produced the best trade-off between reconstruction quality and compression rate. Figure 6 shows the relationship between scaled MSE and compression rate, while Figure 7 shows the combined trade-off score for each configuration.

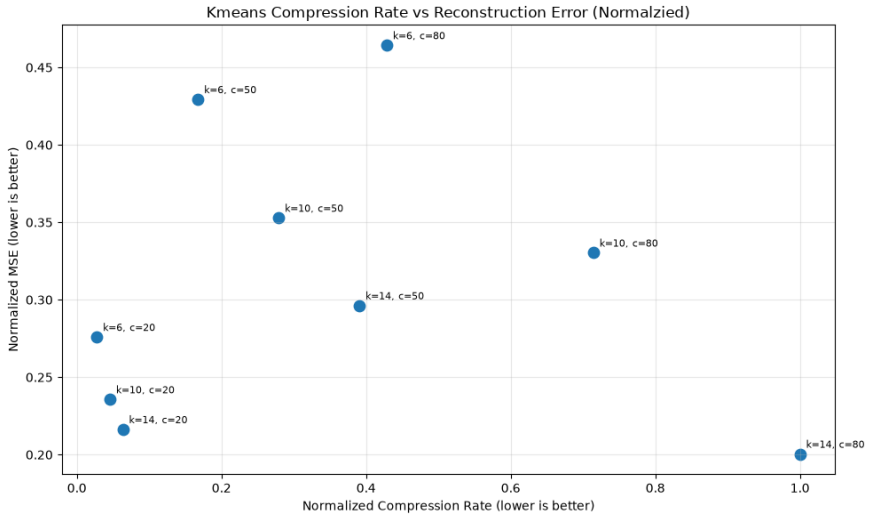


Figure 6: Scaled reconstruction error and compression rate for the k-means configurations. Points closer to the bottom-left represent a better trade-off.

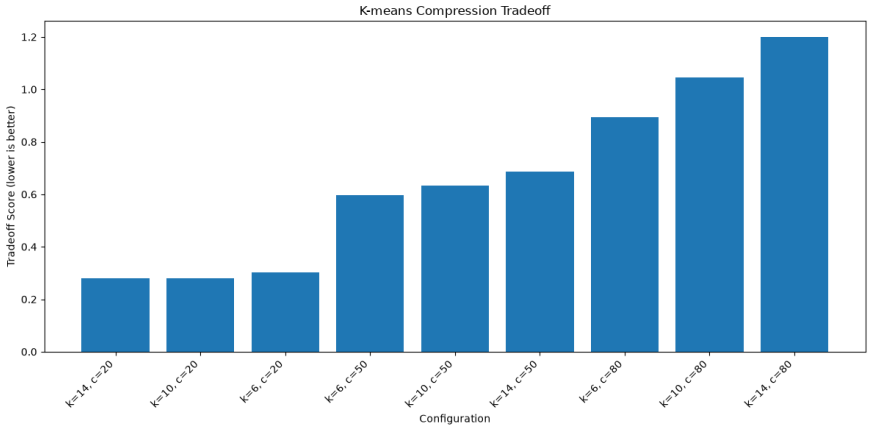


Figure 7: Equal-weight trade-off scores for the k-means configurations. Lower scores indicate a better trade-off.

Overall, k-means produced lower reconstruction error than hierarchical clustering while having the same compression rate for equivalent k and c values. In Figure 8, the k-means results generally lie closer to the lower-left, indicating a better trade-off for this image. This is also visible in the reconstructed images, where the k-means results more closely resemble the original image and preserve more of its color and detail.

This is consistent with the k-means objective, which minimizes within-cluster squared distance around representative centers. That objective aligns with reconstructing each window from a cluster reference vector. Single-linkage hierarchical clustering instead merges clusters using the closest pair of samples and does not directly minimize reconstruction error.

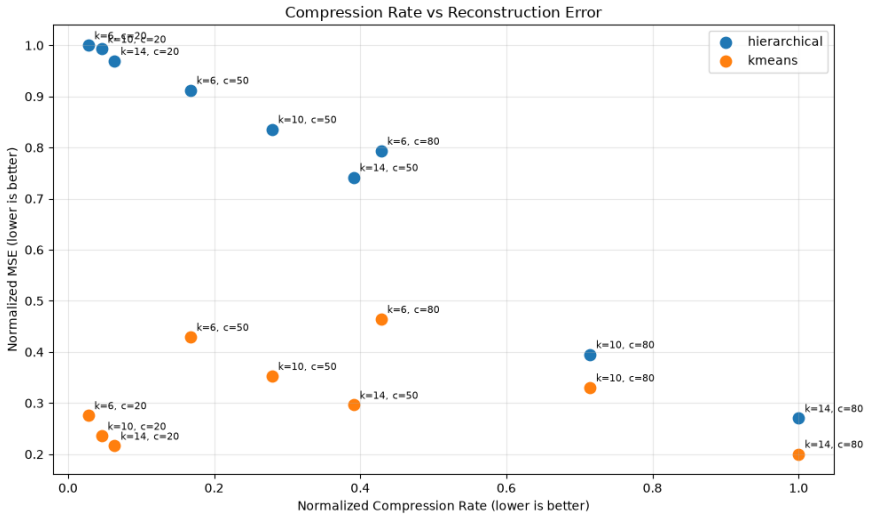


Figure 8: Comparison of the scaled reconstruction error and compression rate for hierarchical and k-means clustering.

Figure 9 compares the combined scores across both methods and further shows the stronger overall trade-off from k-means.

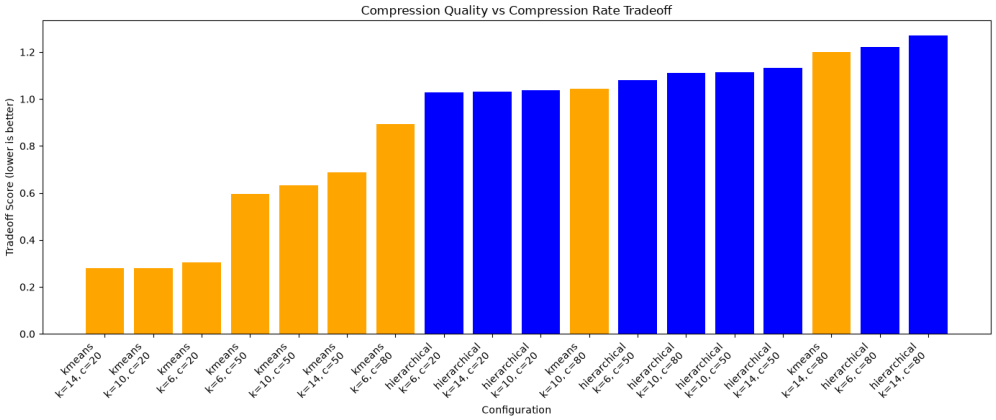


Figure 9: Trade-off scores for all hierarchical and k-means compression configurations. Lower scores indicate a better equal-weight trade-off.

3 Part 2: Iris Dataset Analysis

The Iris dataset contains four attributes: sepal length, sepal width, petal length, and petal width, all measured in centimeters. Each sample belongs to Iris-setosa, Iris-versicolor, or Iris-virginica. This section first estimates the Iris-setosa density with a Gaussian kernel, then compares KNN, a classification tree, and LDA for species classification.

1. Multivariate Kernel Density Estimation

Kernel Density Estimation (KDE) is used to estimate the probability density of the Iris-setosa data without assuming an overall distribution. A Gaussian kernel is centered at each observed sample,

and the kernels are combined to form the density estimate.

For this analysis, only the 50 Iris-setosa samples are used. Since each sample contains four attributes, a four-dimensional multivariate Gaussian kernel is used. For a d-dimensional input, the Gaussian kernel is

$$K(u) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|u\|^2}{2}\right),$$

where

$$u = \frac{x - x^{(t)}}{h}.$$

The resulting multivariate kernel density estimate is

$$\hat{p}(x) = \frac{1}{Nh^d} \sum_{t=1}^{N} K\left(\frac{x - x^{(t)}}{h}\right),$$

where $N = 50$, $d = 4$, and h is the bandwidth.

Three bandwidths, $h = 1.5$, 0.5, and 0.1, are tested. To visualize the four-dimensional estimate, one attribute is varied across its observed range while the other three are fixed at their sample means, producing four plots per bandwidth.

Figures 10–12 show that smaller bandwidths produce sharper peaks and more local variation, while larger bandwidths produce smoother estimates.

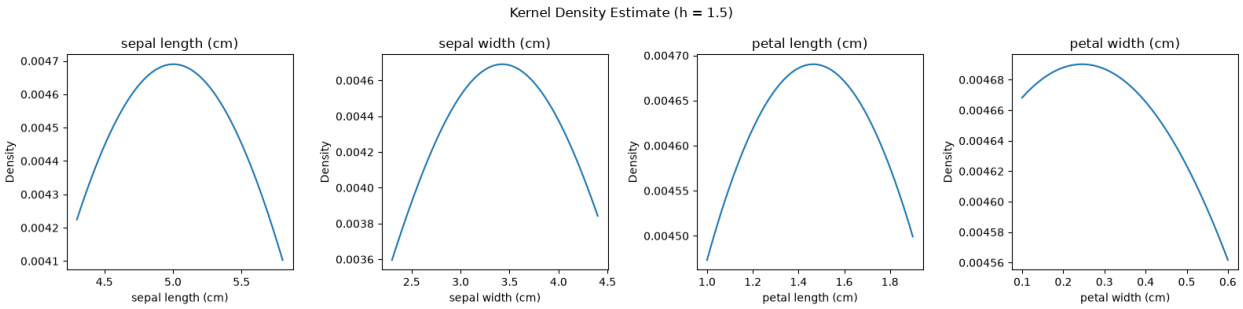


Figure 10: Multivariate KDE slices for the four Iris-setosa attributes using bandwidth $h = 1.5$.

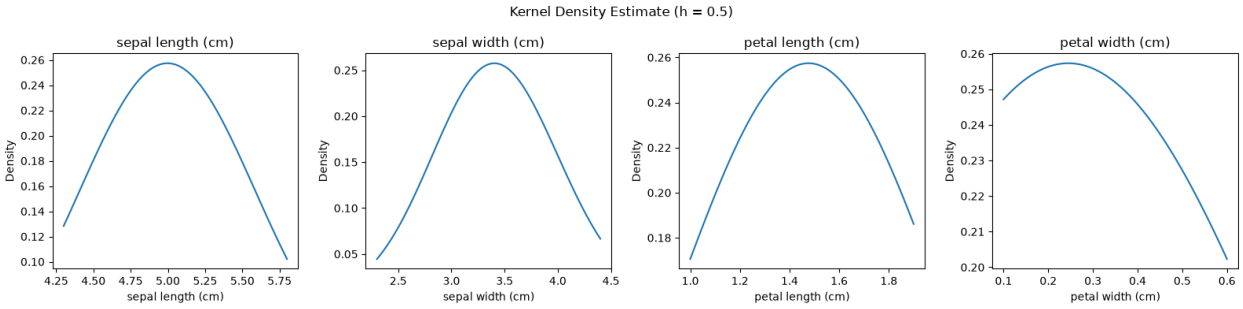


Figure 11: Multivariate KDE slices for the four Iris-setosa attributes using bandwidth $h = 0.5$.

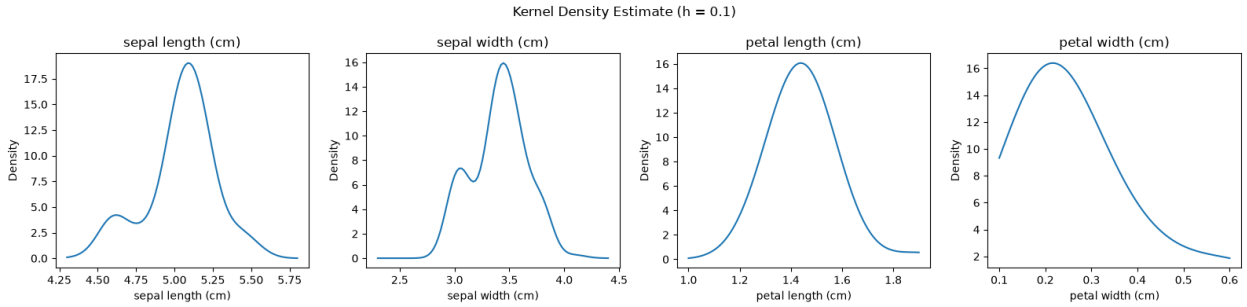


Figure 12: Multivariate KDE slices for the four Iris-setosa attributes using bandwidth $h = 0.1$.

2. Training and Testing Split

The Iris dataset was split into 50% training and 50% testing sets using `train_test_split`. Stratified sampling was used to preserve the proportion of each class in both sets.

3. K-Nearest Neighbors Classification

Using the stratified split, KNN classifiers were trained with $k = 3$, 7, 11, and 15 and evaluated using accuracy and confusion matrices.

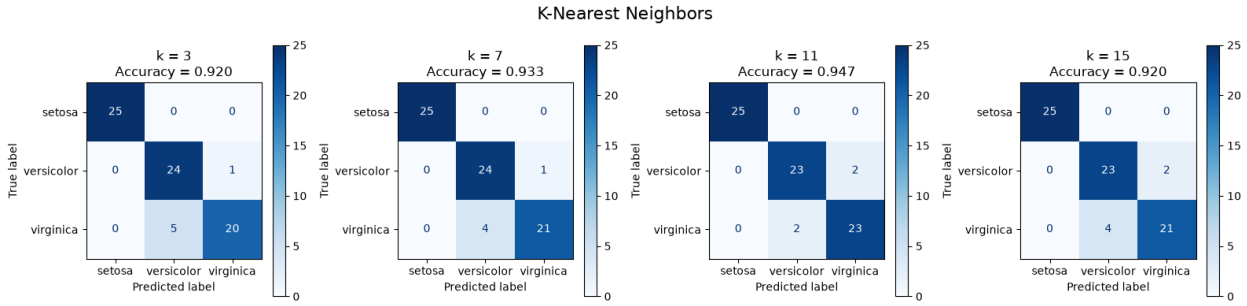


Figure 13: KNN confusion matrices and testing accuracy for $k = 3$, 7, 11, and 15.

As shown in Figure 13, $k = 11$ achieved the highest accuracy at 0.947, with four incorrect predictions. Every model classified all setosa samples correctly; the errors were between versicolor and virginica. Smaller k values are more sensitive to nearby samples, while larger values smooth the decision boundary. Here, performance peaked at $k = 11$ before dropping to 0.920 at $k = 15$.

4. Classification Tree

A classification tree using the Gini index as the impurity measure was trained on the same stratified training set and evaluated on the testing set using prediction accuracy and a confusion matrix.

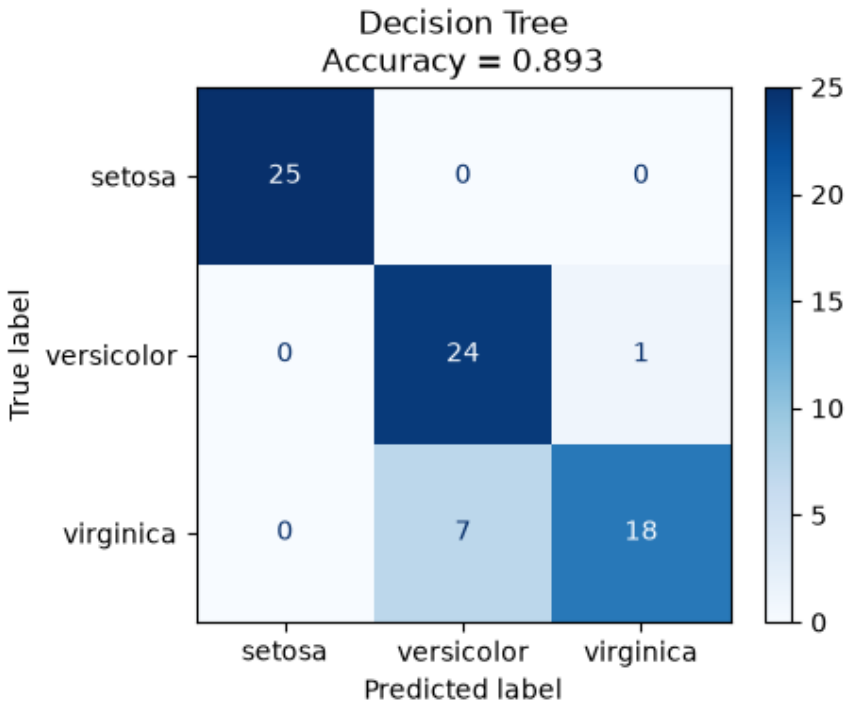


Figure 14: Classification tree confusion matrix and testing accuracy using the Gini index.

As shown in Figure 14, the classification tree achieved an accuracy of 0.893, correctly classifying 67 of the 75 testing samples. All setosa samples were correctly classified. Most errors occurred between virginica and versicolor, with seven virginica samples incorrectly classified as versicolor and one versicolor sample classified as virginica.

5. Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) was trained on the same stratified training set and evaluated on the testing set using prediction accuracy and a confusion matrix.

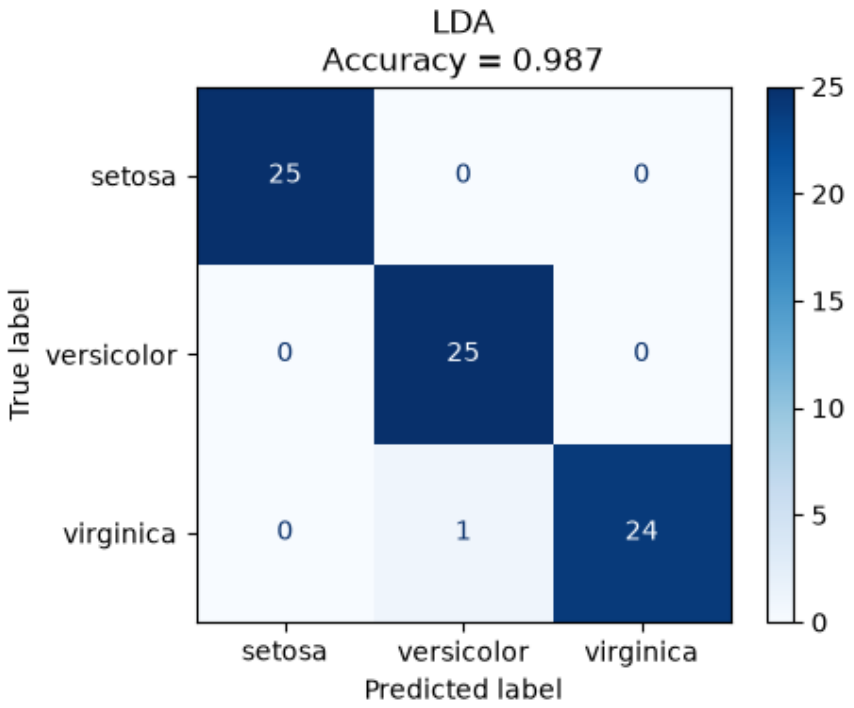


Figure 15: LDA confusion matrix and testing accuracy.

As shown in Figure 15, LDA achieved an accuracy of 0.987, correctly classifying 74 of the 75 testing samples. All setosa and versicolor samples were correctly classified, with the only error being one virginica sample incorrectly classified as versicolor.

6. Classifier Performance Comparison

LDA performed best with 0.987 accuracy and one error, followed by KNN at 0.947 and the classification tree at 0.893. LDA's assumptions appear to fit this dataset well enough for linear boundaries to separate the species effectively. KNN and the tree had more difficulty separating versicolor and virginica, which appear to overlap more in the feature space.

7. Comparison of Classification Methods

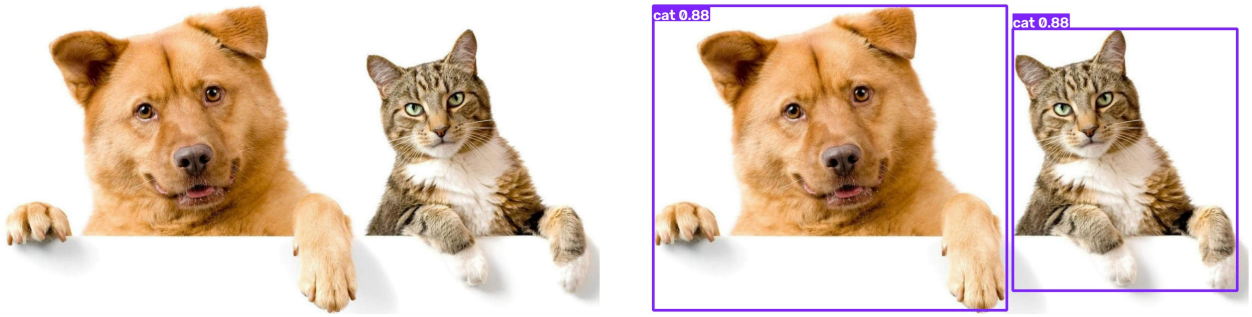
KNN classifies a sample by finding its k nearest training points and using a majority vote. A classification tree uses a sequence of feature-based rules, choosing splits that reduce Gini impurity. LDA models each class with a different mean and a shared covariance matrix, then assigns a sample to the most likely class.

4 Part 3: Neural Networks and Object Recognition

1. Object Recognition with YOLO

A pretrained Ultralytics YOLO26n model is applied to the provided image for object recognition and localization. As shown in Figure 16, the model localizes both animals but labels both as cats.

The cat is classified correctly, while the dog is misclassified. Both detections have a confidence score of 0.88.



(a) Original image (b) YOLO detection results

Figure 16: Original image and object recognition results from the pretrained YOLO model.

2. CNNs and Multilayer Perceptrons

An MLP uses fully connected layers to transform input features into an output. A CNN instead applies learned filters to local regions, allowing it to capture spatial features such as edges, shapes, and more complex patterns.

Both use learned weights, nonlinear activation functions, and backpropagation. The main difference is connectivity: MLP layers use separate weights across their connections, while CNNs share the same filter weights across different input locations. This weight sharing and local structure make CNNs especially useful for images.

5 Part 3: AI and Handwritten Code Comparison

The handwritten and AI-generated implementations produced mostly similar results, with a few differences caused by implementation choices. For image compression with $c = 20$ and $c = 50$, small differences in reconstruction error came from how the cluster representative pixel values were converted back to integers. One implementation rounded the values while the other truncated them. These small pixel differences add up when calculating the total reconstruction error.

Larger differences occurred for $c = 80$ because the image dimensions are not evenly divisible by 80. The AI-generated implementation cropped the image to fit complete windows, while the handwritten implementation used reflection padding and removed the padding after reconstruction. This changed the windows used for clustering and resulted in larger differences in reconstruction error. The compression rates were the same for $c = 20$ and $c = 50$, but differed for $c = 80$ because the two methods used a different number of windows.

The classification and YOLO portions produced the same overall results between the handwritten and AI-generated implementations.